

Greedy Problem Sorting Tips

Yes — **sorting is very common in greedy problems, but it is NOT automatically required.**

The key is to recognize **why** sorting would help.

Main idea

When you see a greedy problem, ask:

"What decision do I want to make first, and does sorting make the best decision easier to identify?"

There are several common patterns.

1. Sort when you need to process "smallest/biggest first"

Example: **Assign Cookies**

You have:

- children's greed: [1, 2, 3]
- cookie sizes: [1, 1]

You want to give the smallest suitable cookie to each child.

So sorting helps:

```
Children:  1  2  3
Cookies:   1  1

1 gets 1
2 cannot get 1
3 cannot get 1
```

Typical solution:

```
C++
sort(g.begin(), g.end());
sort(s.begin(), s.end());
```

Signal

If the problem says something like:

- smallest possible
- largest possible
- minimum resource
- maximum resource
- match elements
- pair elements
- choose the next best element

 **Think about sorting.**

2. Sort intervals when making scheduling decisions

Example: **Non-overlapping Intervals**

```
[1, 2]
[2, 3]
[3, 4]
[1, 3]
```

The greedy decision is:

Which interval should I keep?

The important property is **ending time**.

So sort by ending time:

```
[1, 2]  ← ends at 2
[2, 3]  ← ends at 3
```

```
[1,3] ← ends at 3
[3,4] ← ends at 4
```

Why?

Because an interval that finishes earlier leaves more space for future intervals.

```
Early finish
  ↓
[1-----2]
  [2-----3]
    [3-----4]
```

So:

```
C++
sort(intervals.begin(), intervals.end(),
    [](auto &a, auto &b) {
        return a[1] < b[1];
    });
```

Signal

For interval greedy problems, ask:

What property determines the best interval to choose?

Usually:

- earliest ending time → sort by **end**
- earliest starting time → sometimes sort by **start**
- smallest/largest interval → maybe sort by length

3. Don't sort if the array already gives you the required order

Example: **Jump Game**

```
nums = [2,3,1,1,4]
```

You need to determine whether you can reach the end.

You don't want:

```
C++
sort(nums.begin(), nums.end());
```

✗ That destroys the positional information.

Instead:

```
index:  0  1  2  3  4
nums:   2  3  1  1  4
      ↑
    current
```

Maintain the farthest reachable position:

```
C++
int farthest = 0;
for(int i = 0; i < nums.size(); i++) {
    if(i > farthest)
        return false;

    farthest = max(farthest, i + nums[i]);
}
return true;
```

Signal

If **index/order is important**, be very careful about sorting.

Ask:

"If I reorder the array, does the meaning of the problem change?"

If yes → **probably don't sort.**

4. Sometimes sort is unnecessary because a better greedy structure exists

Example: **Best Time to Buy and Sell Stock**

```
[7, 1, 5, 3, 6, 4]
```

You might think:

"This is greedy, should I sort?"

✗ No.

Because we need:

```
buy BEFORE sell
```

Sorting would destroy that relationship.

Instead:

```
C++
int mn = nums[0];
int profit = 0;

for(int i = 1; i < nums.size(); i++) {
    profit = max(profit, nums[i] - mn);
    mn = min(mn, nums[i]);
}
```

Here the greedy idea is:

Keep the cheapest buying price seen so far.

5. Sort when the problem is about selecting elements

Consider:

Given many activities, attend maximum number of activities.

Example:

```
[1, 4]
[3, 5]
[0, 6]
[5, 7]
[3, 9]
[5, 9]
```

You need to select compatible activities.

Sort by:

```
ending time
```

Then greedily choose the activity that finishes earliest.

This is the classic **Activity Selection** greedy pattern.

6. A very important distinction

When you see a greedy problem, separate these two questions:

Question 1

Does the original order matter?

If yes:

```
Don't blindly sort.
```

Examples:

- Jump Game
- Gas Station
- Best Time to Buy/Sell Stock
- Partition problems where positions matter

Question 2

Does the problem only care about the values/objects, not their original positions?

If yes:

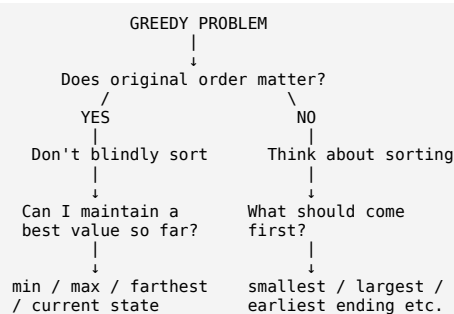
Sorting becomes a strong possibility.

Examples:

- Assign Cookies
- Boats to Save People
- Maximum Units on a Truck
- Meeting/Activity scheduling
- Minimum number of arrows
- Non-overlapping intervals

The most useful greedy detection rule

When you're solving LeetCode, use this mental checklist:



Don't use this rule

Wrong thinking:

"This is greedy → I must sort."

Instead use:

"What makes my greedy choice optimal? Can sorting put the choices in the correct order?"

That's the real question.

Quick examples

Problem	Sort?	Why
Assign Cookies	✓	Match smallest suitable values
Boats to Save People	✓	Pair lightest + heaviest
Non-overlapping Intervals	✓	Earliest ending interval
Merge Intervals	✓	Need intervals ordered by start
Jump Game	✗	Position/order matters
Gas Station	✗	Circular order matters
Best Time to Buy/Sell Stock	✗	Buy must occur before sell
Lemonade Change	✗	Process customers in given order
Maximum Units on Truck	✓	Take highest-value boxes first

★ One sentence to remember

If rearranging the input doesn't change the problem, sorting is often useful; if the original order/indices represent something meaningful, sorting may destroy the information you need.

If you want, I can also give you **the 10 most common Greedy patterns in LeetCode and exactly when to sort for each one.**
ChatGPT can make mistakes. Check important info.